Taylorovy řady

Jak kalkulačka počítá sinus?

Seminář IVT

3. ročník gymnázia

2025

Obsah

1. Motivace
2. Taylorova řada
3. Sinus a kosinus
4. Odmocnina
5. Python

Co dělá kalkulačka, když stisknete sin?

- Zadáme 30°, dostaneme $0{,}5$.

Co dělá kalkulačka, když stisknete sin?

- Zadáme 30°, dostaneme $0{,}5$.
- Jak to vypočítá? Neexistuje „jednoduchý“ vzorec.

Co dělá kalkulačka, když stisknete sin?

- Zadáme $30°$, dostaneme $0{,}5$.
- Jak to vypočítá? Neexistuje „jednoduchý“ vzorec.
- Kalkulačka používá **aproximační algoritmus**.

Dnešní cíl

Naučit se **Taylorovu řadu** — základ pro pochopení aproximací.

Používané metody

- Maclaurin / Taylor
- Čebyšev
- CORDIC
- Tabulky

Metody používané v praxi

Maclaurinovy řady (Taylor v bodě 0)

Nekonečný polynom. Intuitivní, výchozí bod. Pro velká x konverguje pomalu.

Metody používané v praxi

Maclaurinovy řady (Taylor v bodě 0)

Nekonečný polynom. Intuitivní, výchozí bod. Pro velká x konverguje pomalu.

Čebyševovy polynomy

Minimalizují maximální chybu na daném intervalu. Přesnější při stejném počtu koeficientů.

Metody používané v praxi

Maclaurinovy řady (Taylor v bodě 0)

Nekonečný polynom. Intuitivní, výchozí bod. Pro velká x konverguje pomalu.

Čebyševovy polynomy

Minimalizují maximální chybu na daném intervalu. Přesnější při stejném počtu koeficientů.

CORDIC [1]

Jen bitové posuvy a sčítání. Žádné násobení. Vhodný pro jednoduché čipky (digitální hodinky, GPS).

Metody používané v praxi

Maclaurinovy řady (Taylor v bodě 0)

Nekonečný polynom. Intuitivní, výchozí bod. Pro velká x konverguje pomalu.

Čebyševovy polynomy

Minimalizují maximální chybu na daném intervalu. Přesnější při stejném počtu koeficientů.

CORDIC [1]

Jen bitové posuvy a sčítání. Žádné násobení. Vhodný pro jednoduché čipky (digitální hodinky, GPS).

My studujeme Taylor

Konverguje pomaleji, ale je **intuitivní** a je **výchozím bodem** pro

Taylorova řada — idea

Klíč

Každá hladká funkce lze **lokálně přiblížit polynomem**. Čím více členů, tím přesnější aproximace.

Taylorova řada — idea

Klíč

Každá hladká funkce lze **lokálně přiblížit polynomem**. Čím více členů, tím přesnější aproximace.

Maclaurinova řada ($a = 0$):

$$f(x) = f(0) + f'(0) \cdot x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \cdots$$

Taylorova řada — idea

Klíč

Každá hladká funkce lze **lokálně přiblížit polynomem**. Čím více členů, tím přesnější aproximace.

Maclaurinova řada ($a = 0$):

$$f(x) = f(0) + f'(0) \cdot x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \cdots$$

Intuice bez derivací

Člen x = tečna ke křivce v $x_0 = 0$. Člen x^3 = opravuje zakřivení. Každý další člen opravuje chybu předchozího.

$\sin(x)$ — Maclaurinova řada

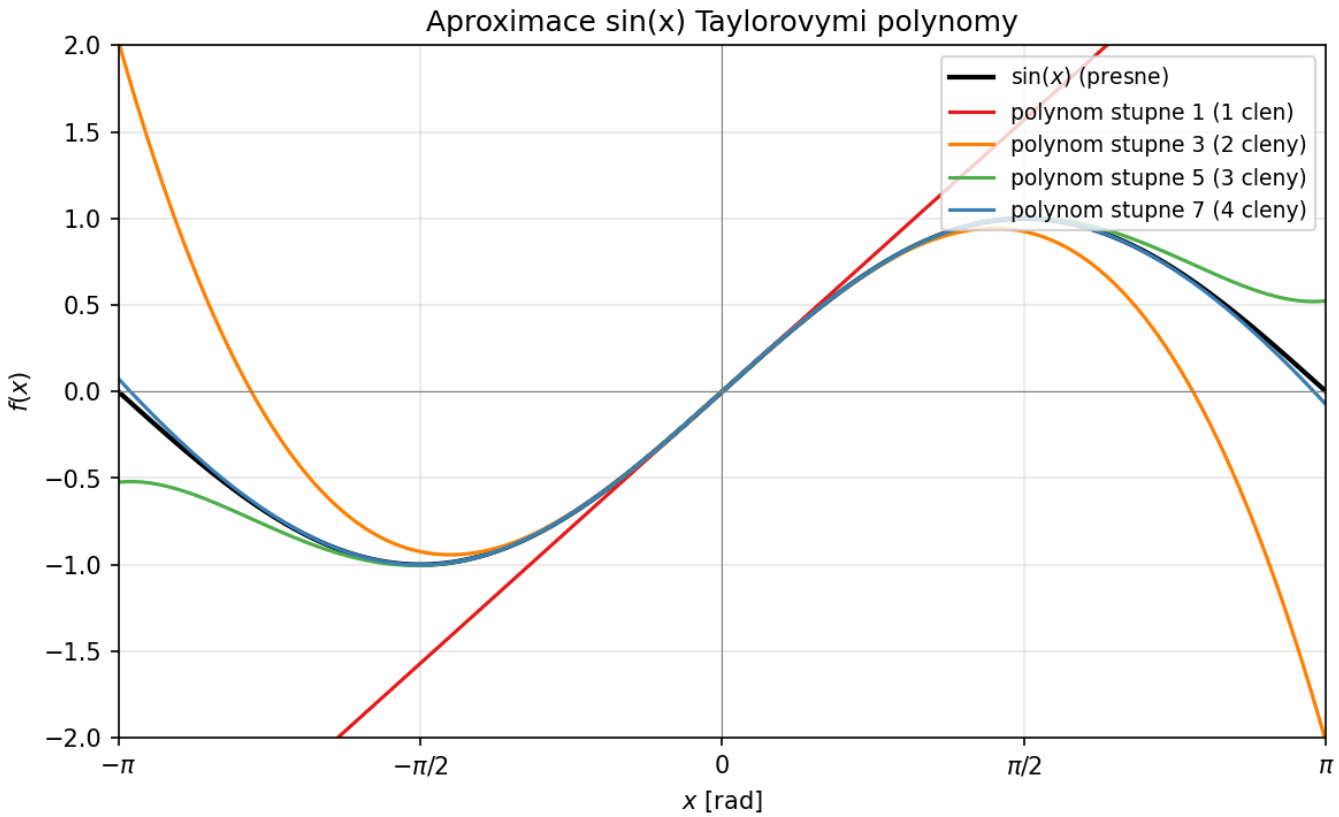
$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Pozor: x musí být v **radiánech**. ($30^\circ = \pi/6 \approx 0{,}5236$ rad)

$\sin(x)$ — Maclaurinova řada

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Pozor: x musí být v **radiánech**. $(30^\circ = \pi/6 \approx 0{,}5236 \text{ rad})$



$\cos(x)$ — Maclaurinova řada

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

$\cos(x)$ — Maclaurinova řada

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Sinus — liché mocniny

$x,\ x^3,\ x^5,\ x^7, \ldots$

Začíná členem x.

Kosinus — sudé mocniny

$1,\ x^2,\ x^4,\ x^6, \ldots$

Začíná členem 1.

$\cos(x)$ — Maclaurinova řada

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Sinus — liché mocniny

$x,\ x^3,\ x^5,\ x^7, \ldots$

Začíná členem x.

Kosinus — sudé mocniny

$1,\ x^2,\ x^4,\ x^6, \ldots$

Začíná členem 1.

Konvergence

Pro $x \in [-\pi, \pi]$ stačí **10 členů** pro přesnost lepší než 10^{-12}.

Odmocnina — Newtonova iterace

Myšlenka: lepší a lepší odhad

1. Počáteční odhad: $x_0 = a$

Odmocnina — Newtonova iterace

Myšlenka: lepší a lepší odhad

1. Počáteční odhad: $x_0 = a$
2. Každý krok:

$$x_{n+1} = \frac{x_n + \frac{a}{x_n}}{2}$$

Odmocnina — Newtonova iterace

Myšlenka: lepší a lepší odhad

1. Počáteční odhad: $x_0 = a$
2. Každý krok:
$$x_{n+1} = \frac{x_n + \frac{a}{x_n}}{2}$$
3. Jen $+$, $\times$ a $/$ — žádné mocniny.

Odmocnina — Newtonova iterace

Myšlenka: lepší a lepší odhad

1. Počáteční odhad: $x_0 = a$
2. Každý krok:

$$x_{n+1} = \frac{x_n + \frac{a}{x_n}}{2}$$

3. Jen $+$, $\times$ a $/$ — žádné mocniny.

Kvadratická konvergence

Počet správných číslic se přibližně **zdvojuje** při každé iteraci!

3700 let stará metoda — tabulka YBC 7289

- Kolem roku **1700 př. n. l.**

3700 let stará metoda — tabulka YBC 7289

- Kolem roku **1700 př. n. l.**
- Babylonský písař zapsal $\sqrt{2}$ na hliněnou tabulku.
- Přesnost: **6 platných číslic**.

3700 let stará metoda — tabulka YBC 7289

- Kolem roku **1700 př. n. l.**
- Babylonský písař zapsal $\sqrt{2}$ na hliněnou tabulku.
- Přesnost: **6 platných číslic**.
- Sexagesimální (základ 60) zápis:
  $1;\ 24,\ 51,\ 10$
  $= 1 + \frac{24}{60} + \frac{51}{3600} + \frac{10}{216000}$
  $\approx 1{,}414\,21$

Stejný algoritmus

Přesně ta iterace,
kterou jsme právě ukázali!

$$x_{n+1} = \frac{x_n + a/x_n}{2}$$

Nezávisle znovuobjevena:
Heron (1. stol. n. l.),
Newton (17. stol.)

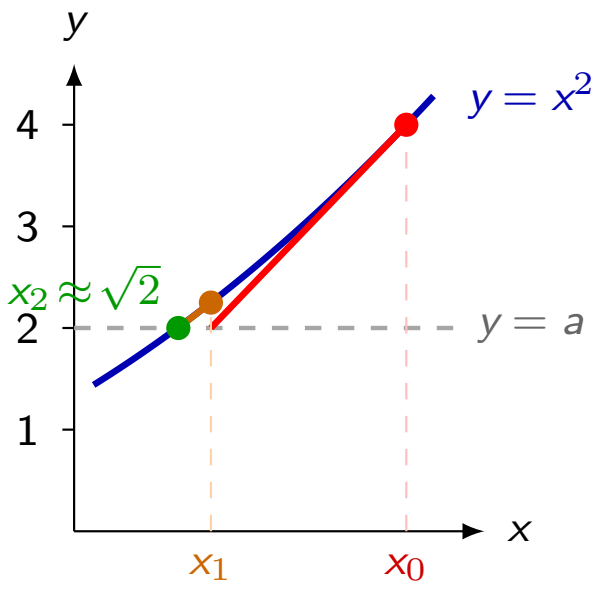
Geometrie: tečna k parabolě

Postup pro $\sqrt{a}$

1. Bod (x_n, x_n^2) na $y = x^2$
2. Tečna protíná $y = a$
3. Průsečík $= x_{n+1}$
4. Opakovat

Tečna v (x_n, x_n^2), sklon $2x_n$:

$$x_{n+1} = \frac{x_n + a/x_n}{2}$$



$\sqrt{2}$ — průchod iterací

Iterace	x_n	Chyba
0	2,000000	$5{,}9 \times 10^{-1}$
1	1,500000	$8{,}6 \times 10^{-2}$
2	1,416667	$2{,}5 \times 10^{-3}$
3	1,414216	$2{,}1 \times 10^{-6}$
4	1,414214	$1{,}6 \times 10^{-12}$
5	1,414214	$< 10^{-15}$

Správná hodnota: $\sqrt{2} = 1{,}414\,213\,56\ldots$

Python: sin_taylor

```python
def faktorial(n):
    vysledek = 1
    for i in range(2, n + 1):
        vysledek *= i
    return vysledek

def sin_taylor(x, n_terms=10):
    soucet = 0.0
    for k in range(n_terms):
        exponent = 2 * k + 1
        clen = ((-1)**k) * (x**exponent) / faktorial(
            exponent)
        soucet += clen
    return soucet
```

Klíč

Jen základní operace. `import math` není potřeba.

Python: sqrt_newton

```python
def sqrt_newton(a, n_iter=10):
    if a == 0:
        return 0.0
    x = a                          # pocatecni odhad
    for _ in range(n_iter):
        x = (x + a / x) / 2
    return x
```

Python: sqrt_newton

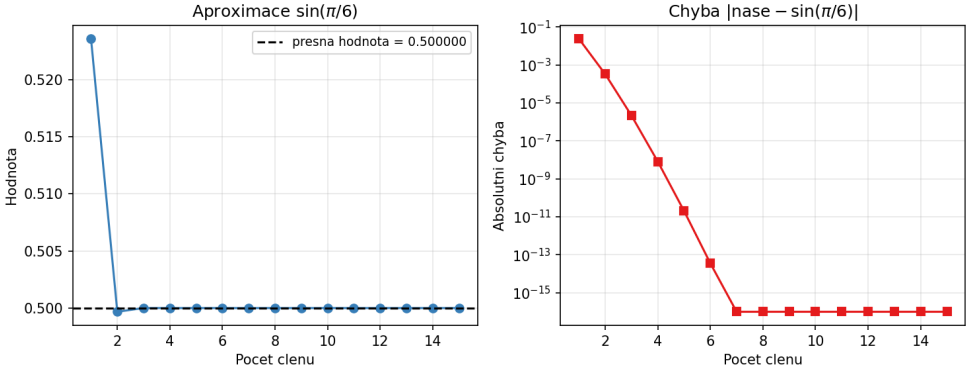
```python
def sqrt_newton(a, n_iter=10):
    if a == 0:
        return 0.0
    x = a                          # pocatecni odhad
    for _ in range(n_iter):
        x = (x + a / x) / 2
    return x
```

Použití

```python
import math
print(sqrt_newton(2, n_iter=5))    # 1.4142135623...
print(math.sqrt(2))                # reference:
    1.4142135623...
```

Tabulka chyb a konvergence

Členů	Chyba $\sin(\pi/6)$
1	$2{,}4 \times 10^{-2}$
2	$3{,}3 \times 10^{-4}$
3	$2{,}1 \times 10^{-6}$
5	$8{,}8 \times 10^{-11}$
7	$< 10^{-15}$



Co se děje v praxi

Redukce oboru (*range reduction*)

Před výpočtem: přivedeme x do intervalu $[0, \pi/2]$ pomocí $\sin(x + 2\pi) = \sin(x)$.
Taylorova řada je pak přesná už při malém počtu členů.

Co se děje v praxi

Redukce oboru (*range reduction*)

Před výpočtem: přivedeme x do intervalu $[0, \pi/2]$ pomocí $\sin(x + 2\pi) = \sin(x)$.
Taylorova řada je pak přesná už při malém počtu členů.

Proč Čebyšev, ne Taylor?

Taylorův polynom stupně n je přesný v $x = 0$, ale na okraji intervalu dělá velké chyby.
Čebyševův polynom rozloží chybu rovnoměrně na celém intervalu.

Co se děje v praxi

Redukce oboru (*range reduction*)

Před výpočtem: přivedeme x do intervalu $[0, \pi/2]$ pomocí $\sin(x + 2\pi) = \sin(x)$.
Taylorova řada je pak přesná už při malém počtu členů.

Proč Čebyšev, ne Taylor?

Taylorův polynom stupně n je přesný v $x = 0$, ale na okraji intervalu dělá velké chyby.
Čebyševův polynom rozloží chybu rovnoměrně na celém intervalu.

CORDIC

Vyhýbá se násobení úplně. Používá se v obvodech bez FPU (GPS, mikrokontroléry, FPGA).

Literatura I

VOLDER, Jack E. The CORDIC Trigonometric Computing Technique. *IRE Transactions on Electronic Computers*. 1959, roč. EC-8, č. 3, s. 330–334.

Shrnutí + domácí úloha

Co jsme se naučili

- Taylorova řada přibližuje funkci polynomem
- $\sin$ a $\cos$: lichými / sudými mocninami s faktoriálem
- Odmocnina: Newtonova iterace (kvadratická konvergence)
- Kalkulačka: redukce oboru + Čebyšev / CORDIC

Shrnutí + domácí úloha

Co jsme se naučili

- Taylorova řada přibližuje funkci polynomem
- $\sin$ a $\cos$: lichými / sudými mocninami s faktoriálem
- Odmocnina: Newtonova iterace (kvadratická konvergence)
- Kalkulačka: redukce oboru + Čebyšev / CORDIC

Domácí úloha (viz zadání)

Implementujte `sin_taylor(x, n)`, `cos_taylor(x, n)`, `sqrt_newton(a, n)` v čistém Pythonu.
Porovnejte s `math.sin` / `math.sqrt` pro alespoň 3 hodnoty.

Děkuji za pozornost!